

دليل تشغيل — تطبيق Fly GACA Pro على iOS (Capacitor + اشتراك Pro)

القسم operations-it-06 النوع runbook الحالة active المالك Founder آخر تحديث 03-07-2026

يُغلف تطبيق iOS تطبيق الويب التقدمي القائم (PWA) داخل غلاف Capacitor ويبيع Fly GACA Pro عبر مشتريات Apple داخل التطبيق. ويعيد استخدام كامل الخلفية: فالشراء يمنح `users/{uid}.entitlement` (تكتبه Cloud Function وحدها)، ويقرأه التطبيق عبر مسار `store.js` / `entitlements.js` نفسه الذي يستخدمه الويب. ما هو موجود بالفعل في المستودع (مبني وآمن على الويب):

- `capacitor.config.ts` — `appld` `com.flygaca.app` ، `webDir: www`
 - `npm run build:ios` (`scripts/build-ios-web.js`) — يجفّع `www/` :يُحزم `parts/` الخاصة بـ GACAR ، `charts/` الخاصة بـ VFR ، والفهارس ، والأدوات ، والدراسة ، والأدلة الإرشادية (~26 ميجابايت)؛ ويستثني أدلة FAA/الأجنبية ، والكتب الإلكترونية ، وفهرس البحث البالغ 19 ميجابايت (تُبثّ + تُخزّن على الجهاز). ويحقن `assets/js/native-bridge.js`
 - `assets/js/native-bridge.js` — رقعة أصلية فقط: توجّه دعوة جدار الدفع إلى ورقة StoreKit ، وشراء/استعادة RevenueCat ، وتخزين جلب مؤقت للأصول المبنوثة. بلا أثر على الويب.
 - `assets/js/auth.js` — فرع تسجيل دخول أصلي عبر Apple/Google (لا يزال الويب يستخدم النافذة المبنوثة).
 - `assets/js/gate.js` — جدار الدفع حيّ فقط داخل التطبيق (`GATING_LIVE`) يكون `true` فقط عندما يكون `Capacitor.isNativePlatform()` ويبقى الويب مفتوحاً.
 - `functions/revenuecatWebhook.js` — RevenueCat ← `users/{uid}.entitlement`
 - `firebase.json` — CORS على `/assets/data/**` كي يستطيع التطبيق بثّ الأصول الثقيلة.
- كل ما يلي يحتاج إلى جهاز Mac مع Xcode + CocoaPods وحسابي Apple Developer RevenueCatg — ولا يمكن إنجازه في CI/Linux.

1. توليد المشروع الأصلي (Mac)

```
npm install --legacy-peer-deps      # .npmrc already sets this
npm run build:ios                    # writes www/
npx cap add ios                      # generates ios/ (Xcode project + Pods)
npx cap sync ios                    # or: npm run cap:sync
npx cap open ios                     # opens Xcode
```

في Xcode: عيّن الفريق/التوقيع، ومعرّف الحزمة `com.flygaca.app` ، وفعل قدرتي Sign in In-App Purchase ، وفعّل وصول المرحلة 2).

2. مصادقة Firebase الأصلية

- أضف iOS app إلى مشروع Firebase flygaca-firebase : نزل `GoogleService-Info.plist` إلى `ios/App/App/`.
- فعل موثري Google و Apple في Firebase Auth (ففعّلان بالفعل للويب). أضف معرف حزمة iOS. لتسجيل الدخول عبر Google، أضف مخطط عنوان URL لمعرف العميل المعكوس إلى هدف iOS.
- تحقق من أشكال طريقة/نتيجة `@capacitor-firebase/authentication` مقابل الإصدار المثبت (v7.x):
`signInWithGoogle()` ، `signInWithApple()` ← `result.credential.{idToken, nonce, accessToken}`
ويبادل الفرع في `auth.js` هذه إلى JS SDK عبر `signInWithCredential`.

3. App Store Connect — منتجات الاشتراك

أنشئ مجموعة اشتراك ذاتية التجديد "Fly GACA Pro" تتضمن:

- `pro_monthly` — 59 ريالاً سعودياً / شهرياً.
- `pro_annual` — 349 ريالاً سعودياً / سنوياً، مع تجربة مجانية 7 أيام (عرض تمهيدي). (تتطابق معرفات المنتجات التي تحتوي `year / annual` مع `source: 'annual'` في الـ `webhook`؛ عدّل `periodOf()` في `functions/revenuecatWebhook.js` إذا اخترت معرفات أخرى). أضف البيانات الوصفية المدّية المطلوبة، ولقطة شاشة المراجعة، وروابط شروط الاشتراك / الخصوصية.

4. RevenueCat

- أنشئ مشروعاً، وأضف تطبيق App Store + السر المشترك الخاص بالتطبيق (App-Specific Shared Secret).
- أنشئ Offering بحزمتي `$rc_monthly` و `$rc_annual` مرتبطين بالمنتجات، و `Entitlement` باسم `pro` مرتبطاً بكليهما.
- انسخ مفتاح SDK العام لـ iOS ← عيّنه للتطبيق. إما مضمناً في `native-bridge.js` (`REVENUECAT_IOS_KEY`) أو اعرض `window.FG_REVENUECAT_IOS_KEY` قبل تحميل ذلك السكربت. إنه مفتاح قابل للنشر، آمن للشحن.
- `Webhook`: وجهه إلى عنوان URL للدالة المنشورة (`https://me-central1-flygaca-` أو `firebase.cloudfunctions.net/revenuecatWebhook` أو عنوان Cloud Run) واضبط ترويسة `Authorization` على سر قوي.
- يستدعي التطبيق `Purchases.logIn(firebaseUid)` بعد تسجيل الدخول عبر `FGNative.setUser()` كي يساوي `app_user_id` معرف `uid` من Firebase.

5. نشر الـ webhook

```
firebase functions:secrets:set REVENUECAT_WEBHOOK_AUTH # same value as the RC header
cd functions && npm ci && cd ..
firebase deploy --only functions:revenuecatWebhook,hosting
```

(يشحن نشر hosting ترويسة CORS الجديدة لـ `/assets/data/**`.)

6. التحقق من حلقة الشراء (وضع الحماية في StoreKit)

1. سجّل الدخول (Apple/Google) على جهاز/محاكٍ.
2. الجأ إلى ميزة محجوبة (مثل أداة طيران رابعة، أو الدراسة) ← يظهر جدار الدفع Go Pro ← اشترِ بحساب وضع الحماية.
3. يُظهر RevenueCat استحقاق `pro` نشطاً ← يكتب الـ webhook الخاص به `users/{uid}.entitlement` (تحقق في Firestore) ← يعيد التطبيق التحميل وُترفع البوابة.
4. سجّل الدخول إلى الويب بالحساب نفسه ← يظهر Pro هناك أيضاً.
5. اختر Restore Purchases والتجربة التمهيدية السنوية.
6. دون اتصال: افتح Part من GACAR وخريطة مع إيقاف الشبكة (محزومة ← تعمل)؛ افتح دليل FAA على الإنترنت مرة، ثم دون اتصال (مبثوث ← مخزن).

قائمة تحقق مراجعة Apple

مشتريات داخل التطبيق فقط لـ Pro (3.1.1) · تسجيل دخول أصلي عبر Sign in with Apple (4.8) · حساب داخل التطبيق + حذف البيانات (موجود بالفعل في `settings.js`، 5.1.1) · Restore Purchases · ملصقات تغذية الخصوصية · أبقِ إخلاء المسؤولية "ليس للاستخدام التشغيلي" بارزاً.

المرحلة 2 — تذكيرات انتهاء الصلاحية + حزم التحضير (في المستودع؛ تحتاج إلى ربط بالجهاز)

تذكيرات انتهاء الصلاحية الدفعية

الشفيرة موجودة بالفعل: `functions/reminders.js` (وظيفة مجدولة يومية `expiryReminders` الساعة 06:00 بتوقيت Asia/Riyadh — تفحص `users`، وتدفع عندما يتجاوز `medicalExpiry` / `lastFlightReview` +24 شهراً عتبة)، و `store.js savePushToken`، و `native-bridge.js registerPush`، وخطاف لوحة المعلومات الذي يسجل عند تسجيل الدخول، للتفعيل:

1. Apple Developer ← أنشئ (p8) APNs Auth Key؛ ارفعه إلى Cloud Messaging ← Firebase Project Settings (iOS).
2. Xcode ← أضف قدرة Push Notifications (Remote notifications ← Background Modes) إلى هدف App.
3. `npx cap sync ios` (تبعية `@capacitor/push-notifications` موجودة بالفعل في `package.json`).
4. النشر: `firebase deploy --only functions:expiryReminders`.
5. التحقق: عيّن ملفاً شخصياً `medicalExpiry` بعد 7 أيام (Settings)، وتأكد من وصول رمز إلى `users/{uid}.fcmTokens`، ثم شغل الدالة "Cloud Scheduler" (Run now) أو المحاكى) وتأكد من وصول الإشعار الدفعي.

حزم التحضير لمرة واحدة (غير قابلة للاستهلاك)

الشفرة موجودة بالفعل: يربط الـ `webhook` `pack_elpt` / `pack_aip` `entitlement.packs[]` (ضمن معاملة، مع الحفاظ على أي خطة Pro)؛ `native-bridge.js purchasePack`؛ ودعوة جدار الدفع توجّه ميزات `pack:*` إليه؛ `gate.js` / `entitlements.js hasPack` يفتحان بالفعل `packs/*.html`.

1. App Store Connect ← أنشئ منتجات غير قابلة للاستهلاك `pack_aip` ، `pack_elpt` (وسّع `PACK_PRODUCTS` في `revenuecatWebhook.js` للمزيد).

2. RevenueCat ← أضف المنتجات؛ يعالج الـ `webhook` `NON_RENEWING_PURCHASE`.

3. التحقق: افتح `packs/aip.html` بصفتك غير مالك ← جدار الدفع ← "Unlock this pack" ← شراء بوضع الحماية ← يضيف الـ `webhook` `aip` إلى `packs[]` ← تفتح الصفحة؛ وتأكد من أن تجديد Pro لاحقاً لا يُسقط الحزمة (تحفظ المعاملة كليهما).

المرحلة 3 — فوتر Stripe على الويب + تراخيص مدارس B2B (في المستودع)

تمرّ الآن كل عمليات كتابة الاستحقاقات عبر مساعد معاملاتي واحد (`functions/entitlements.js`)، تتشاركه مسارات Stripe و Apple والمدارس، كي لا يطمس اشتراك ولا الحزم لمرة واحدة ولا مقعد مدرسة أيّ منها الآخر أبداً.

Stripe على الويب (هامش أعلى من مشتريات Apple داخل التطبيق)

الطيار التجريبي الذي يشترك على الويب يحصل على الاستحقاق نفسه عبر المنصات (Pro في التطبيق أيضاً) دون اقتطاع Apple. يجب ألا يربط التطبيق إطلاقاً أو يذكر دفع الويب (القاعدة 3.1.1) — يرفض `billing.js` بدء الدفع داخل التطبيق. الشيفرة موجودة: `functions/stripe.js` (`createCheckoutSession`) القابلة للاستدعاء + `stripeWebhook` (`assets/js/billing.js`)، `FGBilling.startProCheckout`.

1. Stripe Dashboard ← أنشئ "Product "Fly GACA Pro" بسعرين متكررين: 59 ريالاً سعودياً / شهرياً و349 ريالاً سعودياً / سنوياً.

2. الأسرار: `firebase functions:secrets:set STRIPE_SECRET_KEY STRIPE_WEBHOOK_SECRET` `STRIPE_PRICE_MONTHLY` `STRIPE_PRICE_ANNUAL`.

3. `cd functions && npm i && cd ..` (يضيف حزمة `stripe`)، ثم `firebase deploy --only` `functions:createCheckoutSession,functions:stripeWebhook`.

4. Stripe ← أضف `webhook endpoint` عند عنوان `stripeWebhook` المنشور، مشتركاً في `checkout.session.completed` ، `customer.subscription.created/updated/deleted` . ضع سرّ توقيعه في `STRIPE_WEBHOOK_SECRET`.

5. فعّل دعوة الويب (مهمة إطلاقاً): على `pricing.html` ، حمّل `assets/js/billing.js` واربط دعوة Pro بـ `FGBilling.startProCheckout('annual'|'monthly')` . ولجعل جدار دفع الويب يحجب المشتركين غير المشتركين فعلاً، عيّن `WEB_GATING_LIVE = true` في `assets/js/gate.js` — افعل ذلك فقط بعد أن يصبح الكيان القانوني + الدفع حيّين.

6. التحقق: مستخدم ويب مسجّل الدخول ← الدفع ← بطاقة اختبار ← `checkout.session.completed` ← `Pro = users/{uid}.entitlement` ← سجّل الدخول على iOS، يظهر Pro هناك أيضاً.

تراخيص مدارس B2B (تباع خارج التطبيق)

تشتري المدارس مقاعد بموجب عقد؛ ثم يزود مشغّل الطلاب. الشيفرة موجودة: `functions/school.js` (`grantSchoolLicence` / `revokeSchoolLicence` , للمسؤول فقط).

1. اجعل مشغلاً مسؤولاً (لمرة واحدة): `admin.auth().setCustomUserClaims(uid, { admin: true })` (سكربت Node صغير مع حساب الخدمة، أو صدفّة دالة Firebase console).

2. النشر: `firebase deploy --only functions:grantSchoolLicence,functions:revokeSchoolLicence`.

3. التزويد: استدعِ `grantSchoolLicence({ emails:[...], schoolId, expiresAt })` (مثلاً من أداة مسؤول

أو `firebase functions:shell`). يصبح استحقاق كل طالب `{ plan:'school', source:'school',`

`schoolId, expiresAt }` وهو ما يعامله `isPro()` بالفعل بوصفه وصولاً كاملاً. استخدم

`revokeSchoolLicence` عند نهاية العقد.